Capitolo 10

Processi di sviluppo, Agile, XP e Scrum

10.1 Attività, prodotti e coerenza del processo

Un *processo di sviluppo software* organizza le attività necessarie per ottenere e mantenere un prodotto: stabilisce relazioni, responsabilità, risultati attesi e momenti di verifica. Le fasi possono essere suddivise in compiti, o *task*. Il problema è coordinare il lavoro mentre evolvono bisogni, ambiente e conoscenza del sistema.

Le fasi ricorrenti sono analisi dei requisiti, progettazione, implementazione, convalida e manutenzione. Un *artefatto* è un prodotto intermedio o finale di una fase, per esempio una specifica, un modello UML, codice o una suite di test. La consistenza fra artefatti è importante: ciascuno deve concordare con quelli da cui deriva.

10.1.1 Dalle specifiche all'implementazione

Analisi e specifica trasformano i bisogni in requisiti verificabili; la progettazione assegna responsabilità a sottosistemi, componenti e interfacce; l'implementazione traduce queste decisioni in codice eseguibile. I modelli di oggetti, interazioni, stati e flussi di dati devono restare coerenti con ciò che il programma realizza. Progettazione e implementazione possono alternarsi: un risultato sperimentale può richiedere di rivedere una decisione precedente.

Quando un test rivela un difetto, si localizza la causa, si corregge il modello se esprime una decisione errata, si aggiorna il codice e si riesegue il test. Correggere soltanto il programma lasciando il modello incoerente rende più difficile la manutenzione successiva.

10.1.2 Verifica, rilascio e cambiamento

Verifica e validazione (V&V) accompagnano lo sviluppo mediante revisioni e test a livelli diversi. Il rilascio consegna una versione del prodotto; la manutenzione continua poi con correzioni, miglioramenti e adattamenti. Una modifica può conservare il comportamento, come nel refactoring, oppure aggiornarlo per soddisfare nuovi requisiti o usare nuove piattaforme.

Se il cliente chiede un comportamento diverso dopo aver approvato requisiti già soddisfatti, si analizza la nuova informazione sul bisogno e se ne valuta l'impatto su specifiche, piano e verifiche. La conformità a un documento deve quindi essere accompagnata dalla validazione dell'utilità del prodotto.

10.2 Cascata: stabilità e documentazione delle fasi

Il modello *a cascata* (Waterfall) è un processo sequenziale: si avvia una fase solo dopo aver completato la precedente. La sequenza è definizione dei requisiti, progetto di sistema e software, implementazione e test di unità, integrazione e test di sistema, operazione e manutenzione. Requisiti e documenti delle fasi concluse sono congelati: l'intento è mantenere coerenti tutti gli artefatti.

I vantaggi sono ampia documentazione, consistenza fra artefatti e qualità del codice prodotto. Gli svantaggi sono il tempo lungo per avere il prodotto, poche interazioni con il cliente oltre la fase iniziale e l'elevato costo di introdurre cambiamenti. È indicato quando i requisiti sono stabili e chiaramente definiti, quando occorre gestire gruppi numerosi e per sistemi grandi, complessi o critici, nei quali l'affidabilità è essenziale.

La necessità di processi organizzati nasce anche dalla *crisi del software*: alla fine degli anni Sessanta, attorno al 1969, il rapido progresso dell'hardware conviveva con difficoltà nel rispettare tempi, costi e obiettivi dei programmi. Lo schema sequenziale è associato al lavoro di Royce del 1970 e richiama l'organizzazione di produzioni industriali, come quella automobilistica. Documentazione e coerenza degli artefatti rispondono all'esigenza di governare la complessità, senza garantire da sole la correttezza.

Il richiamo alla legge di Moore e a un raddoppio orientativo su una scala di circa diciotto mesi riassume il rapido progresso hardware. Questo ordine di grandezza non si traduce in un raddoppio periodico garantito della velocità di ogni programma: prestazioni e costo dipendono anche da algoritmi, implementazione e uso delle risorse.

La rappresentazione strettamente sequenziale congela le fasi; uno schema con frecce di ritorno permette invece feedback controllati verso fasi precedenti. Sono due gradi diversi di rigidità: correggere requisiti o progetto è possibile nella seconda rappresentazione, ma può obbligare a riesaminare documentazione e verifiche già approvate. Il modello non va quindi confuso con l'impossibilità materiale di tornare indietro. Una durata complessiva di dieci–dodici mesi è un esempio di pianificazione, non una proprietà necessaria della cascata. Software per aerei o centrali nucleari esemplifica contesti nei quali documentazione, tracciabilità e controllo delle modifiche hanno particolare rilievo.

10.3 Sviluppo evolutivo, incrementale e per riuso

Un *processo evolutivo* costruisce il sistema mediante trasformazioni successive e ha due varianti.

Sviluppo per esplorazione. Sviluppatori e cliente lavorano insieme: si comprende, progetta, implementa e mostra una parte, quindi il feedback guida il passo successivo. È utile quando la visione complessiva non è ancora sufficientemente chiara, ma dovrebbe partire da requisiti iniziali chiari e aggiungere le caratteristiche richieste dal cliente.

Build and Fix. Si costruisce una prima versione e la si modifica fino alla soddisfazione del cliente, con comprensione limitata del sistema e design e documentazione quasi inesistenti. La mancanza di progettazione e documentazione rende difficile controllare la qualità del codice e il costo delle modifiche.

Entrambe le varianti possono richiedere tempi lunghi e produrre sistemi difficili da comprendere, modificare o mantenere corretti, per mancanza di una visione d'insieme. Sono quindi appropriate per sistemi piccoli, singole parti di sistemi grandi, come un'interfaccia utente, o prototipi a vita breve.

Nel flusso evolutivo una descrizione iniziale orienta specifica, sviluppo e validazione, che si alimentano reciprocamente invece di costituire tre blocchi irreversibili. Le versioni iniziale, intermedia e finale rendono visibile questa convergenza progressiva.

Nello *sviluppo incrementale* si implementano per prime le funzionalità di base o prioritarie; a ogni incremento si aggiunge codice per un ulteriore gruppo di funzionalità, fino al completamento. Il processo CBSE (*Component-Based Software Engineering*) basa invece il progetto sul riuso di COTS (*Components Off The Shelf*), cioè componenti esistenti: analisi dei componenti disponibili, eventuale modifica dei requisiti, progetto tramite riuso, sviluppo e integrazione.

Il riuso può ridurre il lavoro di costruzione sia usando COTS sia recuperando componenti già sviluppati dall'azienda. Occorre però verificarne compatibilità, qualità e costo di integrazione. Il prezzo del nuovo progetto non deriva automaticamente da quello dei componenti. Analogamente, lo studio di fattibilità valuta risorse, competenze, tempi e costi, mentre durata e condizioni economiche dello studio dipendono dall'accordo organizzativo.

10.4 Spirale: obiettivi e riduzione dei rischi

Il modello a spirale di Boehm focalizza prodotti parziali, cioè sottosistemi funzionanti. Ogni ciclo della spirale è una fase e attraversa quattro settori: identificazione degli obiettivi della fase corrente; valutazione e riduzione dei rischi; produzione e convalida della parte; revisione del progetto e pianificazione della fase seguente. Un *rischio* è qualcosa di sconosciuto che può impedire il successo, inteso come soddisfacimento dei requisiti; gli attributi rilevanti sono probabilità di occorrenza e impatto, ossia la gravità del danno peggiore. Quando un ciclo realizza una parte del prodotto, il codice sviluppato viene testato e integrato nel sistema complessivo. La ripetizione dei cicli consente una prima versione in tempi ridotti e l'interazione con il cliente.

I cicli possono avere obiettivi diversi: costruire un prototipo, realizzare una parte dell'interfaccia oppure ridurre un rischio tecnico. Esempi di rischio sono la dipendenza da una sola persona competente, l'indisponibilità di personale e difetti nelle librerie. La natura iterativa emerge già confrontando due cicli successivi; una durata dell'ordine di sei mesi per ciclo è un esempio orientativo, da adattare al progetto, non una durata minima del modello.

La spirale è un *metaprocesso*: la valutazione dei rischi orienta anche la scelta del modello e delle tecniche da usare nel settore di sviluppo di ciascun ciclo. Gli obiettivi sono valutati insieme ad alternative e vincoli; simulazioni, modelli e benchmark possono confrontare opzioni prima di impegnarsi nella costruzione. Prototipi successivi, da una prima versione a una terza e poi a un prototipo operativo, riducono incertezze diverse.

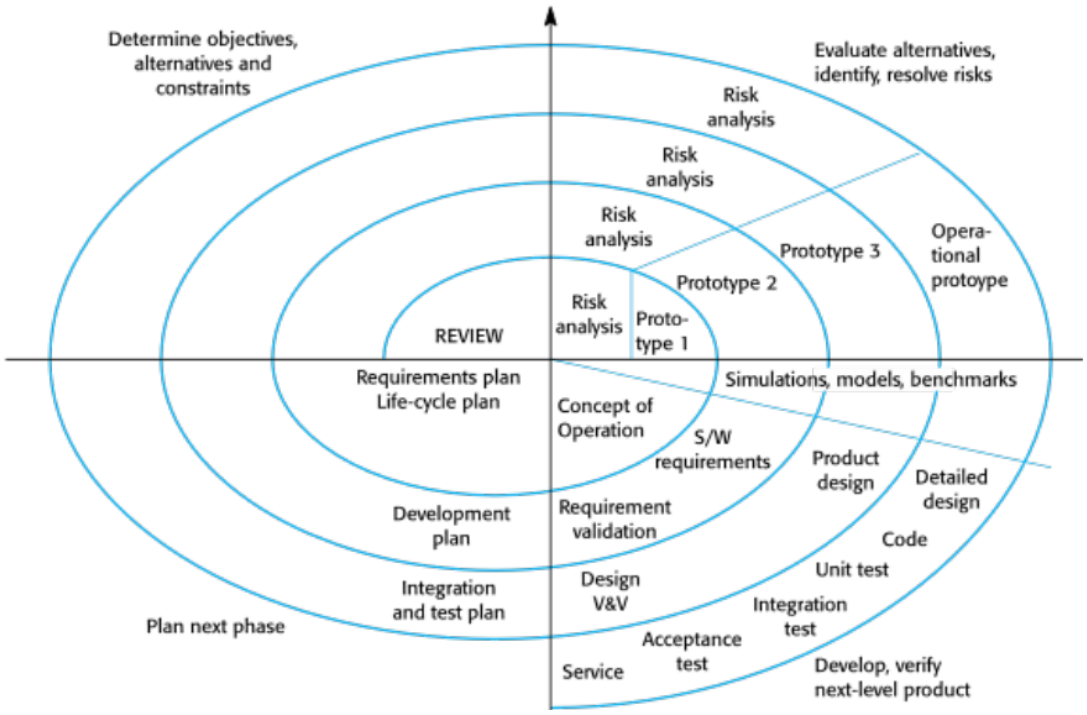


Figura 10.1: Cicli guidati dal rischio: prototipi, verifiche, piani e sviluppo progressivo.

Il dettaglio dei cicli può passare dal concetto operativo ai requisiti software, al progetto del prodotto, al progetto dettagliato, al codice e ai test di unità, integrazione e accettazione. Validazione dei requisiti e verifica e validazione del progetto accompagnano tali passi. Piani del ciclo di vita, dello sviluppo e dell'integrazione e dei test collegano gli impegni dei cicli successivi. Questa varietà spiega perché non ogni giro debba ripetere identiche attività né produrre subito codice definitivo.

Se si ipotizzano almeno due cicli di circa sei mesi, il calendario dell'esempio è dell'ordine di un anno; si tratta di una scelta di scala, non di un minimo matematico del modello. In quel contesto una revisione complessiva dei rischi non coincide con una semplice riunione settimanale o mensile. La frequenza appropriata dipende comunque dall'entità del rischio e dalla durata effettiva dei cicli.

10.5 Approccio Agile: valori, feedback e cambiamento

Il Manifesto per lo sviluppo Agile del 2001 privilegia individui rispetto a processi e strumenti, software funzionante rispetto alla documentazione, collaborazione con il cliente rispetto alla negoziazione contrattuale, e pronta risposta ai cambiamenti rispetto alla mera esecuzione di un piano. Il confronto non dichiara inutili gli elementi a destra: indica che, nelle scelte, hanno maggiore valore quelli a sinistra.

L'agilità accoglie positivamente richieste di cambiamento anche avanzate, fornisce frequentemente release funzionanti, costruisce il sistema con persone motivate e presta continua attenzione all'eccellenza tecnica. Collaborazione, comunicazione e adattabilità richiedono persone motivate, competenze adeguate e disponibilità del cliente a fornire chiarimenti e feedback frequenti.

10.6 Extreme Programming: dodici pratiche che si sostengono

Extreme Programming (XP), formalizzato nel 1999, è un metodo Agile basato sullo sviluppo e sulla consegna di piccoli incrementi di funzionalità. Usa iterazioni corte e di durata costante, piccoli gruppi, indicativamente da due a dodici sviluppatori, comunicazione diretta fra sviluppatori, clienti e manager, miglioramento costante del codice e prodotti testati fin dall'inizio. Privilegia una documentazione essenziale e aggiornata durante il lavoro, in particolare story card e CRC card. È adatto a requisiti instabili e a progetti con rischi elevati, per esempio consegne brevi o software innovativo per il team.

I suoi principi sono feedback rapido, semplicità, cambiamenti incrementali, supporto al cambiamento e produzione di lavoro di qualità.

Tabella 10.1: Le dodici pratiche di Extreme Programming.

Pratica	Funzione nel lavoro quotidiano
Gioco di pianificazione	Cliente e sviluppatori definiscono storie, stime e priorità; le storie eccessive vengono suddivise.
Pair programming	Una postazione condivisa, ruoli driver e navigator, revisione continua e rotazione dei partner.
Piccole release	Parti piccole ma funzionanti permettono feedback rapido; una cadenza tipica è di due settimane.
Metafora	Un'immagine condivisa aiuta a discutere architettura, nomi e scelte.
Design semplice	Si costruisce la soluzione necessaria ora, guidata da test, chiarezza e assenza di duplicazione.
Testing	Test di unità e funzionali documentano il comportamento e proteggono l'integrazione.
Possesso collettivo	Tutti sono responsabili del codice e possono migliorarlo, rispettando standard e test.
Integrazione continua	Le parti testate vengono integrate frequentemente e verificate insieme.
Settimana di quaranta ore	Un ritmo sostenibile evita che la stanchezza comprometta ragionamento e qualità.
Cliente in sede	Una persona disponibile chiarisce storie, decide priorità e definisce risultati attesi.
Standard di codifica	Convenzioni comuni su nomi, formattazione e commenti rendono il codice uniforme.
Refactoring	Piccole trasformazioni migliorano il progetto conservando il comportamento verificato.

Le dodici pratiche si sostengono reciprocamente: per esempio, la proprietà collettiva del codice richiede standard condivisi e test affidabili, mentre integrazione frequente e refactoring beneficiano di modifiche piccole. Omettere una pratica può indebolire i benefici delle altre.

10.6.1 Gioco di pianificazione, story card e piccole release

Nel *gioco di pianificazione*, gli utenti scrivono le *storie utente* sulle *story card*; gli sviluppatori stimano il tempo necessario a ciascuna storia. Una storia troppo complessa per essere stimata deve essere riportata al cliente e divisa. Gli utenti dividono, fondono e assegnano priorità alle

storie; gli addetti al business decidono date delle release, contesto e priorità. La pianificazione dell'intera release è grossolana e quella della nuova iterazione è ravvicinata: non si pianifica troppo in avanti. Per la release corrente, gli sviluppatori scompongono ogni storia in task, li stimano e si impegnano a realizzarli, affrontando prima i task più rischiosi.

Una story card è un caso d'uso leggero: in due o tre frasi, importante per il cliente, testabile, stimabile e associabile a una priorità. I campi indicati sono data, numero, priorità, tempo stimato, riferimenti, descrizione del requisito, lista delle attività e note. Le *piccole release* devono essere il più piccole possibile e richiedono tipicamente due settimane: consentono feedback rapido, riducono il rischio, rendono visibili risultati validi e permettono aggiustamenti ai requisiti.

Le story card possono avere il formato di schede di circa 12×7 cm: poco spazio incoraggia richieste piccole, comprensibili e verificabili. Data e numero identificano le schede; stime e priorità rendono esplicita la selezione. Una storyboard può avere colonne per prima settimana, seconda settimana e lavoro successivo. La misura fisica e le colonne sono strumenti dell'esempio, non requisiti del metodo.

Story card: un browser di immagini

Per il progetto *Gestione Immagini*, tre storie scritte da Jim, tutte con priorità 1, separano comportamenti verificabili. Jack ne stima rispettivamente tre, uno e due giorni:

1. mostrare i file JPG e PNG di una cartella in una griglia di 10×6 immagini, indipendentemente dalla risoluzione dello schermo;
2. ridimensionare le immagini rispettandone le proporzioni iniziali;
3. selezionare le immagini digitando lettere in una casella di testo: restano quelle il cui nome di file contiene il testo inserito.

La terza scheda reca la data 15 marzo 2011. Autore, stimatore, stima, sviluppatori e tempo effettivamente impiegato sono campi diversi: confrontarli permette di apprendere dall'esecuzione. Una board su tre settimane distribuisce le schede nelle aree *To do* e *Done*, rendendo visibile ciò che resta e ciò che è stato completato. Le durate sono stime di quel lavoro, non limiti universali per una storia.

Nel planning game il cliente ordina le storie soprattutto per valore; l'indicazione di non farsi guidare inizialmente dalle dipendenze riguarda questa priorità di business. Gli sviluppatori devono comunque analizzare le dipendenze quando scompongono, stimano e ordinano i task. La stima si discute collettivamente: anche due giorni possono risultare troppi per una specifica iterazione e motivare una suddivisione, senza definire una soglia generale. Una terza area per il lavoro successivo evita di fingere una pianificazione dettagliata di tutto il futuro.

10.6.2 Pair programming, metafora, design semplice e CRC

Nel *pair programming* due programmatori lavorano insieme: il *driver* usa tastiera e mouse e ragiona sull'implementazione del metodo; il *navigator* valuta l'approccio, lo revisiona, pensa ai test e cerca una soluzione più semplice. Ruoli e partner si scambiano. La pratica diffonde la conoscenza del sistema e richiede motivazione e collaborazione; un forte divario di competenze può ridurne l'efficacia se impedisce a entrambi di partecipare al ragionamento.

Nella configurazione con una sola postazione, il navigator non lavora in parallelo su un secondo PC: segue il codice e può annotare su carta i test da eseguire. La rotazione permette di condividere conoscenza e controllo del risultato, invece di dividere stabilmente chi scrive e chi osserva.

Una *metafora* guida il progetto con una frase che rappresenta l'architettura e rende più semplici le discussioni; per esempio, "l'interfaccia utente è un desktop". Deve risultare

comprensibile al cliente. Il *design semplice* è quello che passa i test, non contiene parti duplicate, esprime le intenzioni importanti per i programmatori e usa il minor numero possibile di classi e metodi. La regola è fare la cosa più semplice che può funzionare e pagare solo ciò che si usa, evitando di progettare anticipatamente caratteristiche non richieste.

Le CRC card (*Class–Responsibility–Collaboration*) descrivono il design in linguaggio naturale: il nome della classe compare in alto, seguito da due aree per responsabilità e collaborazioni. Sono uno strumento per ragionare su compiti e relazioni, anziché una lista di firme Java.

Schede, pair programming, codice leggibile, test e cliente disponibile distribuiscono la conoscenza del progetto durante il lavoro. La documentazione non richiede necessariamente una fase separata, ma deve comunque comprendere ciò che serve per usare, verificare e mantenere il prodotto.

Dalle storie alle responsabilità: tre CRC card

Le stesse storie conducono alle seguenti schede. La distinzione fra responsabilità e collaboratori impedisce di trasformare ogni requisito in una classe isolata.

Classe	Responsabilità	Collaborazioni
`ImgBrowser`	Mostrare un frame con due pannelli, uno per le immagini e uno per i controlli.	`ImgPanel`, `CtrlPanel`, `JFrame` come superclasse.
`ImgPanel`	Visualizzare le immagini, occupare la parte superiore della finestra e calcolare le proprie dimensioni.	`Cartella`, `Img`, `JPanel` come superclasse.
`Cartella`	Leggere i file immagine da una cartella, filtrarli secondo un criterio e conoscere il numero di immagini presenti e selezionate.	`Img`, `File` di `java.io`.

La superclasse è indicata come tale: non è una collaborazione fra due oggetti distinti. La griglia e le dimensioni riguardano la presentazione; lettura, selezione e conteggio dei file restano nella responsabilità della cartella. Le schede preparano così le interfacce e le collaborazioni prima dei corpi dei metodi.

10.6.3 Testing, integrazione, proprietà collettiva e ritmo sostenibile

In XP si testa continuamente tutto ciò che può fallire e si rieseguono i test più volte al giorno dopo nuovo codice. I *test funzionali*, scritti dal cliente dal punto di vista dell'utente, sono automatizzati, eseguiti almeno ogni giorno e costituiscono parte della specifica eseguibile dei requisiti. Gli *unit test*, scritti dagli sviluppatori, possono precedere il codice nel TDD (*Test Driven Development*) o seguirlo e sostengono design, codifica, refactoring e qualità.

Il *possesso collettivo del codice* significa che ciascuno può modificare ogni parte del sistema ed è responsabile dell'intero sistema; gli unit test proteggono le funzionalità. L'*integrazione continua* integra il codice testato ogni poche ore, al più entro un giorno. Tutti gli unit test devono passare; se falliscono, la coppia che ha prodotto il codice deve ripararlo oppure scartarlo e ricominciare.

Le ulteriori pratiche sono settimana di quaranta ore, per mantenere la capacità di risolvere problemi e non introdurre difetti dovuti alla stanchezza; cliente in sede, che scrive i test funzionali, stabilisce priorità, risponde alle domande e fornisce il contesto; standard di codifica condivisi, affinché il codice sia uniforme e leggibile; refactoring in piccoli passi, protetto dai test, per migliorare la struttura senza alterare il comportamento.

Con dodici sviluppatori organizzati in sei coppie, una storia stimata in quattro ore consente di integrare più volte nella giornata. La suite globale può però diventare molto grande: occorre distinguere test unitari rapidi e controlli più costosi, senza rinunciare alla verifica dell'integrazione. Gli straordinari ripetuti sono un segnale per riesaminare stime, carico, impedimenti e pianificazione frequente; non una soluzione strutturale alla scarsa produttività. Le quaranta ore esprimono il ritmo sostenibile della pratica, da leggere nel contesto organizzativo.

Gli standard rendono esplicite scelte concrete: spazi o tabulazioni per l'indentazione, posizione delle parentesi graffe, convenzioni sui nomi e uso dei commenti. L'obiettivo è leggere e modificare codice scritto da qualunque membro del gruppo, non imporre una formattazione universalmente migliore.

10.6.4 Strumenti di collaborazione e condivisione della conoscenza

Wiki, sistemi domanda–risposta e strumenti di condivisione della sessione di lavoro possono aiutare a conservare e distribuire conoscenza. La tecnologia wiki, associata al lavoro di Ward Cunningham, permette la modifica collaborativa delle pagine; Stack Overflow, attivo dal 2008, organizza domande e risposte tecniche. Questi strumenti affiancano il lavoro del gruppo senza sostituire test e comprensione del sistema.

Il *mob programming*, diffusosi come pratica riconoscibile dal 2014, estende la collaborazione dalla coppia a un gruppo: per esempio, circa cinque partecipanti possono comprendere sviluppatori, cliente e architetto. Le sessioni condivise da remoto, divenute particolarmente rilevanti dal 2020, possono usare strumenti come Live Share. Numeri e configurazioni sono esempi organizzativi: il criterio è permettere ai partecipanti di contribuire e condividere il ragionamento.

Dal 2023 l'uso di ChatGPT come supporto alla programmazione aggiunge una possibile interazione con strumenti generativi alle forme storiche di collaborazione. Il programmatore resta responsabile di comprendere, integrare e verificare il codice: un suggerimento automatico non svolge da solo le responsabilità del navigator né sostituisce la conoscenza condivisa del team.

10.7 Scrum: empirismo, lean e responsabilità del team

Scrum, descritto da una guida pubblicata inizialmente nel 2010, è una struttura di base per generare soluzioni a problemi complessi. Un *Product Owner* elenca il lavoro nel *Product Backlog*; lo *Scrum Team*, composto da Developers, Product Owner e Scrum Master, trasforma una parte del lavoro in un *incremento* durante uno *Sprint*, un'iterazione di durata fissata, al massimo un mese. Il Product Backlog è l'elenco ordinato del lavoro relativo al prodotto. Team e *stakeholder*, ossia le persone interessate al prodotto, valutano il risultato e apportano gli aggiustamenti per lo Sprint successivo.

Scrum è volutamente semplice e incompleto: altri processi e tecniche, inclusa XP, possono essere usati al suo interno. Si fonda su *empirismo*, per cui conoscenza e decisioni derivano dall'esperienza osservata, e su *lean*, cioè riduzione dello spreco e concentrazione sull'essenziale. L'approccio iterativo e incrementale mira a prevedibilità e controllo del rischio.

Il Product Owner risponde del valore del prodotto e della gestione del backlog; i Developers pianificano il lavoro e costruiscono l'incremento; lo Scrum Master sostiene l'efficacia di Scrum. Queste responsabilità chiariscono chi cura obiettivi, realizzazione e funzionamento del processo senza imporre le pratiche tecniche specifiche di XP.

10.7.1 Trasparenza, ispezione e adattamento

I tre pilastri sono:

Trasparenza. Processo e lavoro sono visibili a chi li svolge e a chi riceve i risultati. Le decisioni importanti si basano su Product Backlog, Sprint Backlog e incremento.

Ispezione. Artefatti e progresso verso l'obiettivo sono esaminati frequentemente e diligentemente per individuare problemi; gli eventi Scrum ne fissano la cadenza.

Adattamento. Se processo o risultato diventano inaccettabili, devono essere corretti immediatamente per evitare ulteriori divergenze.

10.7.2 Product Goal, Sprint Goal, backlog e Definition of Done

Il backlog può comprendere lavoro funzionale e non funzionale e continua a evolvere con ciò che il gruppo apprende. Ogni artefatto ha un impegno che ne rende esplicito lo scopo: Product Goal per il Product Backlog, Sprint Goal per lo Sprint Backlog e Definition of Done per l'incremento.

Il *Product Goal* descrive lo stato futuro del prodotto ed è associato al Product Backlog. Lo *Sprint Goal* è l'obiettivo dello Sprint. Lo *Sprint Backlog* comprende lo Sprint Goal, gli elementi del Product Backlog scelti per lo Sprint e il piano per realizzarli; un *incremento* è un risultato che avvicina al Product Goal. La *Definition of Done* precisa lo stato dell'incremento quando soddisfa le misure di qualità richieste; chiarisce quindi che cosa significhi davvero "finito", anche per obiettivi intermedi.

Done deve indicare un risultato osservabile: per esempio codice integrato, test superati e qualità richiesta verificata. Scrivere l'ultima riga di codice non basta se la Definition of Done richiede anche altre attività. I criteri di accettazione descrivono invece le condizioni specifiche della funzionalità.

Empirismo e lean si traducono così in decisioni fondate su risultati osservabili e attenzione al lavoro necessario. L'esempio Toyota rende concreta la riduzione degli sprechi: rifornire i pezzi secondo le necessità evita l'accumulo di grandi quantità in magazzino.

10.7.3 Eventi e cadenze

Gli eventi sono lo Sprint, tipicamente di un mese; lo *Sprint Planning*, che pianifica il lavoro e dura al massimo otto ore per uno Sprint mensile; il *Daily Scrum*, incontro giornaliero di quindici minuti per ispezionare il progresso e adattare lo Sprint Backlog; la *Sprint Review*, che ispeziona i risultati e decide gli adattamenti; e la *Sprint Retrospective*, in cui il team migliora qualità ed efficacia esaminando persone, interazioni, processi, strumenti e Definition of Done.

Per uno Sprint mensile la Sprint Review dura al massimo quattro ore ed è il penultimo evento: segue la Retrospective, che conclude lo Sprint. Per Sprint più brevi le durate sono normalmente ridotte. Il Daily si svolge alla stessa ora e nello stesso luogo per ridurre la complessità organizzativa; resta un incontro di quindici minuti per adattare il lavoro verso lo Sprint Goal, non una revisione finale del prodotto.

10.8 Esempio: gestione degli ordini con XP e Scrum

Si consideri una funzionalità di gestione degli ordini. Nel processo XP il cliente può scrivere una story card: "un utente inserisce un ordine e ottiene la conferma", assegnandole priorità e criteri osservabili per un test funzionale. Gli sviluppatori stimano la storia; se troppo ampia, per esempio perché include pagamento, controllo disponibilità e spedizione, chiedono di dividerla. Nella release corrente una coppia realizza prima il task più rischioso, scrive ed esegue unit test, integra il codice solo quando i test passano e mostra la piccola release al cliente. Se il feedback richiede una variazione, il requisito viene ri-prioritizzato nella pianificazione successiva.

Lo stesso lavoro in Scrum può essere registrato nel Product Backlog. Durante lo Sprint Planning il team seleziona l'elemento, formula lo Sprint Goal e lo pone nello Sprint Backlog. L'incremento è accettabile soltanto quando soddisfa la Definition of Done concordata, per esempio implementazione, test e integrazione. Daily Scrum, Review e Retrospective consentono rispettivamente di adattare il lavoro corrente, valutare il risultato con gli stakeholder e migliorare il modo di lavorare. Questo esempio combina i due metodi senza confonderli: Scrum organizza il ciclo, XP suggerisce come programmare e verificare.

10.9 Dominio, specifiche e strumenti di IA

Nel *DDD*, sviluppo guidato dal dominio, la comprensione del contesto applicativo orienta il modello a oggetti. Per individuare entità, significato dei dati e responsabilità non basta conoscere il linguaggio di programmazione: occorre comprendere il problema che il software deve rappresentare. Le scelte e i pattern legati al dominio dipendono quindi dalle caratteristiche di quel contesto.

Lo *SDD* (*Specification Driven Development*) pone al centro le specifiche, anche per guidare strumenti di intelligenza artificiale che producono parti dell'implementazione. La precisione della richiesta è essenziale, e il codice generato deve essere confrontato con le specifiche mediante verifiche e test. L'applicabilità dipende dal sistema: nei progetti grandi, risultati inventati dai modelli linguistici di grandi dimensioni (LLM, *Large Language Models*), detti *allucinazioni*, possono compromettere la coerenza del lavoro. Disporre di un algoritmo generato non risolve da solo i problemi di architettura, integrazione e verifica.

Dominio, specifiche e test possono contribuire allo stesso sviluppo: chiariscono rispettivamente il contesto del problema, il comportamento richiesto e le evidenze della sua realizzazione. Un'eventuale evoluzione di Scrum verso una maggiore integrazione dell'IA resta una prospettiva distinta dall'adozione di strumenti nel lavoro quotidiano: da sola non definisce nuovi ruoli, eventi o artefatti.